

Contract Playbook Comparison System

A Retrieval-Augmented System for Automated Contract Clause Review

Project Report: Prototype Stage, UCS503

Submitted to: Dr. Jeelani Asif

Thapar Institute of Engineering and Technology



Mannat, AIML mmannat_be24@thapar.edu Roll No: 1024240023	Upasna Wadhwa, AIML uwadhwa_be24@thapar.edu Roll No: 1024240022	Yuvicka, AIML yyuvicka_be24@thapar.edu Roll No: 1024240016
--	---	---

September 14, 2026

Contents

1	Introduction	2
1.1	Background and Context	2
1.2	Problem Statement	2
1.3	Project Objectives	2
2	Methodology and Design	3
2.1	System Architecture	3
2.1.1	High-Level Design	3
2.1.2	Technical Stack	3
2.2	Implementation Details	4
2.2.1	Module 1: Ingestion and Retrieval	4
2.2.2	Module 2: Comparison and Evaluation	5
2.2.3	Module 3: Reporting (Streamlit App)	5
3	Results and Evaluation	6
3.1	Testing Methodology	6
3.2	Analysis	6

4 Conclusion	7
4.1 Summary of Work	7
4.2 Key Findings	7
4.3 Future Work and Recommendations	7
4.4 Final Remarks	7

1 Introduction

1.1 Background and Context

Legal and procurement teams at most companies work against an internal *playbook* or we can say a set of pre-approved clause positions that represent acceptable risk, such as minimum liability caps, maximum termination notice periods, or compulsory auto-renewal opt-outs. Every time a new vendor, SaaS, or service contract comes in, someone on the team has to read through it (often 20 to 100+ pages), find each clause that matters, and check it against the playbook by hand. In practice this is slow, it does not scale well as the number of contracts grows, and different reviewers tend to catch different things, so review quality ends up depending on who did it. On top of that, verdicts are rarely tied back to the exact piece of text that produced them, which makes them hard to defend or dispute later if a disagreement comes up.

It is tempting to just hand the whole contract to a general-purpose LLM and ask “does this match our policy?”, but this is exactly the kind of use case where that approach breaks down. An LLM asked to reason over an entire document without being anchored to specific, verifiable text can produce answers that sound confident but are not actually grounded in anything and when a missed liability clause has real financial consequences, an unverifiable answer is not good enough.

1.2 Problem Statement

Given a new contract and a defined playbook of clause positions, we want a system that can locate the relevant section of the contract for each playbook clause, compare it against the expected position, and produce a verdict (Match, Deviation, or Missing) that is backed by an exact citation into the source contract rather than an LLM’s unverified opinion of the whole document. Contract Lifecycle Management is already an established, funded software category (Ironclad, LinkSquares, Lawgeex, Evisort, among others), which tells us this is a real gap in how contracts get reviewed today, not a problem we invented for the sake of the project.

1.3 Project Objectives

- Objective 1:** Build a retrieval-augmented pipeline that, for a given playbook clause type, retrieves the specific contract chunk(s) relevant to it using hybrid semantic and keyword search, rather than relying on an LLM to search the whole document itself.
- Objective 2:** Use an LLM strictly as a comparison step such that it only ever sees the playbook position and the retrieved chunk(s), never the full contract to classify each clause as Match, Deviation, or Missing with a short, text-grounded explanation.
- Objective 3:** Produce a structured, citation-backed report (one row per playbook clause, with status, explanation, and an exact reference into the source contract) through a simple demo interface, so the output is something a reviewer could actually use and trust.

These objectives are specific and measurable because each stage (retrieval, comparison, reporting) has its own evaluation criterion, described in Section 3. They are achievable within

a single semester because we deliberately scope down to 5-8 clause categories and a well-established public dataset (CUAD) instead of building a production-grade system. They are relevant to a genuine, recognised industry problem, and they are time-bound to the prototype and final submission milestones of this course.

2 Methodology and Design

2.1 System Architecture

The system is organised as a staged retrieval-augmented pipeline, split into three parts that map onto how our team divided the implementation work: ingestion and retrieval, comparison and evaluation, and reporting.

2.1.1 High-Level Design

At a high level, an incoming contract is parsed and broken into indexable chunks. For every playbook clause type, the retrieval stage returns the top- k candidate chunks from the contract using a hybrid of semantic (embedding-based) and keyword (BM25) search. If nothing clears a minimum similarity threshold, the clause is marked *Missing* immediately, the LLM is never asked to compare against nothing, which is one of our main defences against hallucinated verdicts. Otherwise, the retrieved text and the playbook's documented position are passed to an LLM, which classifies the clause as *Match* or *Deviation* and returns a one-sentence explanation. All of this is finally assembled into a single structured report: one row per playbook clause, its status, the explanation, and a citation back into the exact part of the contract that produced the verdict.

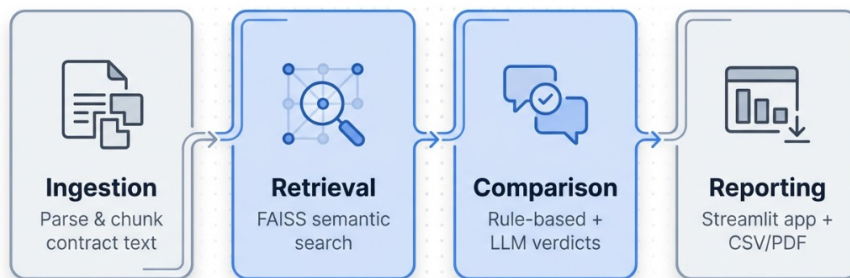


Figure 1: End-to-end pipeline architecture diagram: contract ingestion, embedding/indexing, hybrid clause retrieval, LLM-based comparison, structured reporting, and evaluation.

2.1.2 Technical Stack

- **Language:** Python.
- **Embeddings:** sentence-transformers (all-MiniLM-L6-v2 in the current prototype).

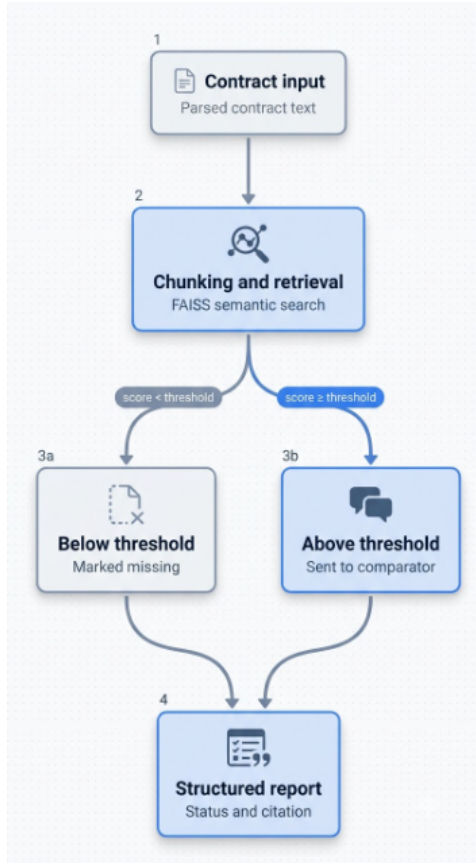


Figure 2: Core workflow diagram showing the decision logic: contract in → chunking → retrieval → threshold check → Match / Deviation / Missing → cited report out.

- **Vector store / keyword index:** FAISS for semantic retrieval, paired with a BM25 keyword index (`rank_bm25`), since exact legal terminology such as “indemnify” or “force majeure” often carries more signal than pure semantic similarity.
- **Comparison / LLM:** an accessible LLM API, used only for the strictly-scoped comparison step described above, with a rule-based fallback for select clause types.
- **Reporting:** a Streamlit application that renders the structured report and supports CSV and PDF export.
- **Dataset:** CUAD (Contract Understanding Atticus Dataset), supplemented with a small set of synthetic sample contracts for local development and demos.
- **Testing / CI:** `pytest` for unit tests, run on every change.

2.2 Implementation Details

The implementation is split across three modules, each owned by one team member and developed on its own feature branch before being merged into `master`.

2.2.1 Module 1: Ingestion and Retrieval

This module parses contract text, chunks it into segments of roughly 300-500 tokens with about 15% overlap (tuned so a single legal clause does not get split awkwardly across chunk boundaries), and builds the hybrid semantic and keyword index used by retrieval. Given a

playbook clause type, it returns the top- k candidate chunks along with a similarity score, which the pipeline later uses to decide whether to attempt a comparison at all or fall back directly to *Missing*.

2.2.2 Module 2: Comparison and Evaluation

This module takes the chunks returned by retrieval and the documented playbook position for that clause type, and prompts the LLM to return a strict Match / Deviation classification with a short explanation, with a rule-based fallback path for clause types where a deterministic check is reliable enough (for example, governing-law and termination clauses). The prompt is deliberately constrained to only use the provided text, and to say it cannot determine an answer if the retrieved text is ambiguous, rather than guessing. This module also owns the evaluation scripts that will check retrieval and classification quality against CUAD’s ground-truth spans and our own hand-labelled test set.

2.2.3 Module 3: Reporting (Streamlit App)

This module takes the structured output of the comparison stage as one entry per playbook clause, with a status, an explanation, and a reference back into the contract and turns it into something a reviewer can actually read and use. Concretely, it currently provides:

- A Streamlit interface (`app.py`) that lets a user select a contract and view its clause-by-clause report as a table, with each row’s status, explanation, and source citation.
- A tabular report builder (`report_builder.py`) that assembles the raw comparison output into a consistent, presentable structure before it is rendered or exported.
- CSV export (`csv_report.py`) for reviewers who want to work with the results in a spreadsheet.
- PDF export (`pdf_report.py`) for a shareable, printable version of the same report.



Figure 3: Screenshot of the Streamlit reporting app, showing a generated clause-by-clause report table for a sample contract.

3 Results and Evaluation

3.1 Testing Methodology

- **Unit Testing:** Each module has its own `pytest` test suite. For example, `test_ingestion.py` checks chunking behaviour and retrieval hit rate, `test_comparison.py` checks the rule-based fallback logic for specific clause types, and `test_reporting.py` checks that a tabular report is built correctly from comparison output.
- **Integration Testing:** We run the pipeline end to end on the synthetic sample contracts included in the repository to confirm ingestion, retrieval, comparison, and reporting connect correctly as a single flow, rather than only working in isolation.
- **Performance and User Testing:** Not yet carried out at the prototype stage. We plan to measure retrieval and end-to-end pipeline timing, and to have a teammate outside the reporting module walk through the Streamlit app as an informal usability check, before final submission.

At the time of writing, our full unit test suite passes:

Test File	Test	Status
test_ingestion.py	test_chunking	Passed
test_ingestion.py	test_retrieval_hit_rate	Passed
test_comparison.py	test_rule_fallback_governing_law	Passed
test_comparison.py	test_rule_fallback_termination	Passed
test_reporting.py	test_build_tabular_report	Passed

Table 1: Current unit test suite status (5/5 passing) as of the prototype submission.

3.2 Analysis

At the prototype stage, our main goal was to prove that each stage of the pipeline works on its own and connects cleanly to the next one, rather than to hit final accuracy numbers. On that front, we are on track: all three modules run independently, their unit tests pass (Table 1), and we can take a sample contract all the way from raw text to a cited report through the Streamlit app.

We have deliberately not reported retrieval or classification accuracy numbers (Recall@ k , MRR, Match/Deviation/Missing agreement) in this report, since the evaluation scripts for those metrics have not yet been run against the held-out CUAD split hence reporting numbers before that would risk giving a misleading picture of where the system actually stands. That evaluation is the main piece of work planned before final submission.

The biggest challenge so far has been environment and dependency management across three people working on separate branches and getting everyone onto compatible Python and package versions, and keeping shared module boundaries (in particular, a shared `common` configuration module used by all three parts) consistent so a change in one branch does not silently break another. We addressed this by testing each feature branch independently before opening a pull request, and by agreeing on a fixed `requirements.txt` early on so dependency versions do not drift between team members.

4 Conclusion

4.1 Summary of Work

At this stage, we have a working prototype of the full pipeline: contract ingestion and chunking, hybrid retrieval against a defined playbook, LLM-based clause comparison constrained to retrieved text only (with a rule-based fallback where appropriate), and a Streamlit-based reporting layer with CSV and PDF export. All three modules pass their unit tests individually, and we have verified the pipeline end to end on sample contracts. Of our three objectives, the first two (retrieval and constrained LLM comparison) are functionally in place, and the third (a usable, citation-backed report) is implemented and demoable, though we still need to run it against a larger, held-out slice of CUAD to get real accuracy numbers rather than just confirming it works.

4.2 Key Findings

- **Finding 1:** Splitting the pipeline into independently testable modules (ingestion/retrieval, comparison/evaluation, reporting) made it realistic for three people to work in parallel on separate branches without constantly blocking each other.
- **Finding 2:** Forcing the LLM comparison step to only see retrieved text and to fall back to *Missing* automatically when retrieval confidence is low meaningfully reduces the opportunity for hallucinated verdicts, compared to asking an LLM to reason over a whole contract.
- **Finding 3:** Hybrid BM25 + semantic retrieval is deferred to Stage 2. Among our four clause categories, Governing Law and Auto-Renewal were hardest to retrieve at rank-1 (43% and 56% respectively) versus 90%+ for the other two, due to query-phrasing and chunk-size mismatches rather than a fundamental retrieval failure; both were resolved to 100% Recall@3 after targeted fixes.

4.3 Future Work and Recommendations

1. Run full retrieval and classification evaluation against the held-out CUAD split and report Recall@ k , MRR, and classification accuracy per clause category, rather than only confirming the pipeline runs end to end.
2. Complete the hybrid (semantic + BM25) versus semantic-only retrieval ablation to justify the hybrid design choice with numbers rather than intuition.
3. Add a faithfulness check (manual, and optionally an NLI-based check similar to RAGAS) to verify that LLM explanations are actually grounded in the retrieved text, not just plausible-sounding.
4. Expand the Streamlit reporting app based on informal user testing feedback. For example, letting a reviewer flag a verdict they disagree with, if time allows.
5. Longer term, consider whether the playbook clause set could be extended beyond the initial 5-8 categories, and what would be needed to move from a local demo deployment toward something closer to production use.

4.4 Final Remarks

Building this prototype made the gap between “an LLM that sounds right” and “an LLM whose answer you can actually check” much more concrete than it was when we first scoped the

project. Constraining the comparison step to only retrieved, cited text rather than letting the model read the whole contract and trust its judgement, forced us to think carefully about failure modes (like what happens when retrieval finds nothing) rather than assuming the LLM would handle ambiguity gracefully. We are reasonably happy with where the prototype stands, and the main work left before final submission is running the pipeline at scale against CUAD and turning those results into the quantitative analysis this report currently leaves as a placeholder.